HealthNexus admin guide

How an administrator uses HealthNexus day to day, followed by a reference of every admin feature and the files that implement it.

Demo accounts

All three use the password **HealthNexus@2026**. They live in the development database only.

Role	Email	What is seeded
Admin	`admin.chennai@admin.example.com`	3 pending worker applications (one with documents to review, one flagged for a reused file, one with none), 3 support concerns, a Chennai advisory, outbreak clusters, 190 centres
PHC worker	`phc.worker@example.com`	Mylapore PHC: 28 citizens, open flags, visits, a Mandaveli fever cluster
Citizen	`citizen.demo@example.com`	Mylapore PHC: smartwatch readings, flags, visits, alerts, health-worker examinations

The three pending applicants (`lakshmi.applicant@`, `arun.applicant@`, `deepa.applicant@example.com`) also use the same password, so you can see the pending-approval screen from their side.

The admin workflow

1. Start at the overview

Sign in as the admin and you land on **Overview** (`/admin`). The top line says what is waiting on you: pending applications and open concerns. The tiles below show workers with no health centre (they see no citizens), open flags across all centres, accounts by role and the last 7 days of activity. The sidebar badges on **Applications** and **Concerns** count the same things and update while you work.

2. Review PHC worker applications

Open **Applications** (`/admin/applications`, filtered to *Pending*). For each applicant:

1. Read the claim: employee ID, designation, the centre they say they work at, district and years of service.
2. Check **Documents**. Each upload has already been checked automatically:
    - *Checks passed*: OCR found the employee ID and the name.
    - *Needs a closer look*: the same file was used by another applicant, it was saved by an image editor, or OCR found neither the ID nor the name.
    - *Waiting for review*: a PDF, or OCR is not installed on the server.
3. Press **View** to open the file (each view is written to the audit trail), then **Accept** or **Reject** it with a reason.
4. If there are no documents, the panel says so: you are deciding from the typed details alone.

3. Approve and link, reject, or revoke

- **Approve:** the registry's exact match for the claimed centre is pre-selected. Change it with the picker if needed, add an optional note and press *Approve and link*. The worker can now see that centre's citizens. The applicant must have confirmed their email first.
- **Reject:** give a reason; the applicant sees it on their pending page.
- **Revoke** (for an approved worker): give a reason; their booked visits are cancelled and the citizens are told.

Every decision appears in the application's history and in the audit trail.

4. Keep the health centre registry right

Open **Health centres** (`/admin/centres`). Look for:

- centres marked **not yet checked** (seeded, not verified): open *Edit*, correct the details and save to mark them checked;
- centres with **0 workers** (shown in amber): their citizens have nobody reviewing readings;
- centres with **No location**: add latitude and longitude so they appear on maps.

Use **Add a centre** for a new facility. **Deactivate** stops new citizens and applicants choosing a centre; people already linked keep their link.

5. Watch for outbreaks and warn people

Open **Outbreaks** (`/outbreaks`). Clusters come from DBSCAN over the last 14 days of symptomatic readings (re-run every 6 hours, or press **Run now**). Each card says how many people, how close together, over which days, and how that compares with the same area's usual level:

- **Possible outbreak**: more cases than chance would explain ($p < 0.01$).
- **Watch**: a group worth keeping an eye on.

Press **Mark as seen** so other admins and workers know it is handled, or **Write an advisory** to open a pre-filled draft.

6. Publish community advisories

Open **Advisories** (`/advisories`). Choose **One district** or **Everyone in the state**, write a short title and plain message, pick how urgent it is and how long it should show (3 to 90 days), then **Publish**. The card shows how many citizens it reaches. **Withdraw** removes it from citizens' Alerts pages at once.

7. Answer support concerns

Open **Concerns** (`/admin/concerns`). Move each from *Open* to *In review* to *Resolved* or *Rejected*, with a note the person can read in their Settings → Support tab.

8. Manage accounts

Open **Accounts** (`/admin/accounts`) to find anyone by name or email and filter by role or status. **Deactivate** (with a reason) signs the person out everywhere at once; **Reactivate** restores access. Administrator accounts cannot be changed here.

9. Check the audit trail

Open **Audit trail** (`/admin/audit`) to see every sensitive action, newest first: approvals, rejections, revocations, document views and decisions, deactivations, registry edits, advisories, research exports and examinations recorded by health workers.

10. Export research data

On the **Overview**, the *Research data* card downloads a CSV of de-identified readings from citizens who turned on research use, and nobody else. There are no names, contact details, exact ages, areas or times of day. Each person gets a code that changes with every download, and every download is audited.

11. Ask the assistant

Assistant (`/assistant`) answers plain questions with aggregates only, never individual readings. For example: "How many applications are pending?", "Which centres have the most flags?", "Activity by district this month", "Workers with no centre". With a Gemini key configured it words the answers; the queries it runs are fixed and read-only.

Feature and file reference

Everything an administrator can do, feature by feature, with the files that implement it. Paths are from the repository root. Every admin API route is under `/api/v1` and requires an admin account (`require_admin` in `backend/app/core/dependencies.py`); the frontend hides admin pages behind `ProtectedRoute allowedRoles={["admin"]}` in `frontend/src/main.tsx`.

#	Feature	Admin page
1	Console overview	`/admin`
2	PHC worker applications	`/admin/applications`
3	Verification documents and OCR checks	inside an application
4	Accounts	`/admin/accounts`
5	Health centre registry	`/admin/centres`
6	Outbreak signals (DBSCAN)	`/outbreaks`
7	Community advisories	`/advisories`
8	Support concerns	`/admin/concerns`
9	Audit trail	`/admin/audit`
10	Research data export	`/admin` (card)
11	Assistant for admins	`/assistant`
12	Navigation, access and seed accounts	

1. Console overview

The landing page for admins: what is waiting (pending applications, open concerns), workers with no centre, open flags across every centre, account counts by role, and activity over the last 7 days.

Layer	File
API	`GET /admin/overview` in `backend/app/api/routes/admin.py`
Page	`frontend/src/pages/admin/AdminDashboard.tsx`
Client	`getOverview` in `frontend/src/services/admin.ts`
Sidebar badges	`useBadges` in `frontend/src/components/app/AppShell.tsx` (pending applications, open concerns, with a toast when a new one arrives)

2. PHC worker applications

A health worker signs up with an employee ID, designation and the centre they claim. Until an admin approves them they have the `pending_phc` role and see no health data. The admin reviews the claim against the registry, links the worker to a centre (approve), rejects with a reason, or revokes an approved worker (their booked visits are cancelled and the citizens told).

Layer	File
API	`GET /admin/applications`, `GET /admin/applications/{id}`, `POST .../approve`, `POST .../reject`, `POST .../revoke`, `GET /admin/facilities` (picker) in `backend/app/api/routes/admin.py`
Registry match	`match_facility`, `normalise_facility_name` in `backend/app/services/phc_registry.py`
Models	`backend/app/models/phc_application.py`, `backend/app/models/phc_verification_audit_log.py`
Page	`frontend/src/pages/admin/Applications.tsx` (list, review panel, facility picker, decision history)
Applicant side	`frontend/src/pages/PHCSignup.tsx`, `frontend/src/pages/PendingApproval.tsx`
Tests	`backend/tests/test_admin.py`

3. Verification documents and OCR checks

Applicants upload an employee ID card, appointment order or registration certificate (JPEG, PNG or PDF, up to 5 MB) from the pending-approval page. Files are kept in private storage and are only ever served to admins. Each upload is checked automatically; the checks never approve anyone, they sort documents for the admin:

- the same file already sent with another person's application: flagged
- saved by an image editor (EXIF software tag): flagged
- OCR finds the employee ID and the applicant's name: checks passed
- OCR ran but found neither: flagged
- PDFs, or no Tesseract on the server: waiting for manual review

The admin views each file (logged in the audit trail) and accepts or rejects it with a reason.

Layer	File
API (applicant)	`GET/POST /phc-application/documents`, `DELETE /phc-application/documents/{id}` in `backend/app/api/routes/applications.py`
API (admin)	`GET /admin/applications/{id}/documents/{doc}/file`, `POST .../decision` in the same file
Checks and storage	`backend/app/services/documents.py`
Model	`backend/app/models/phc_verification_document.py`
UI	`frontend/src/components/app/VerificationDocuments.tsx` (`ApplicantDocuments`, `ReviewDocuments`), client in `frontend/src/services/documents.ts`
Settings	`PRIVATE_UPLOAD_DIR`, `TESSERACT_CMD` in `backend/app/core/config.py` and `backend/.env.example`
Tests	`backend/tests/test_documents.py`

4. Accounts

Every account with filters by role and status and a search. Deactivating signs the person out everywhere at once and needs a reason; administrators cannot be changed from here.

Layer	File
API	`GET /admin/users`, `POST /admin/users/{id}/deactivate`, `POST /admin/users/{id}/reactivate` in `backend/app/api/routes/admin.py`
Sessions	`revoke_all_for_user` in `backend/app/services/sessions.py`
Page	`frontend/src/pages/admin/Accounts.tsx`
Tests	`backend/tests/test_admin.py`

5. Health centre registry

The list of centres citizens choose from and worker applications are matched against. Admins add, edit, deactivate and reactivate centres. The list shows how many workers and citizens are linked to each, and flags centres with no workers or no map location. A deactivated centre cannot be chosen by new sign-ups; people already linked keep their link. Editing a seeded centre marks it as checked.

Layer	File
API	`GET/POST /admin/registry`, `PATCH /admin/registry/{id}`, `POST .../deactivate`, `POST .../reactivate` in `backend/app/api/routes/registry.py`
Model	`backend/app/models/valid_phc_facility.py`
Page	`frontend/src/pages/admin/Centres.tsx`, client functions `getCentres`, `createCentre`, `editCentre`, `setCentreActive` in `frontend/src/services/admin.ts`
Seeds	`backend/app/seeds/seed_phc_facilities.py`
Tests	`backend/tests/test_registry.py`

6. Outbreak signals (DBSCAN)

Every 6 hours the backend looks at the last 14 days of readings from citizens who allowed community monitoring:

1. Each reading is mapped to the syndromes it points to: febrile (fever, body ache, rash, or a temperature of 38 °C or more), respiratory (cough, breathlessness, chest pain, or SpO_2 of 93 % or less) and gastrointestinal.
2. Per syndrome, each person counts once, at their first onset in the window.
3. Cases are projected to kilometres with time as a third axis (one day counts as 0.5 km) and clustered with DBSCAN: `eps` 1.5 km, `min_samples` 3. Three is also the smallest group any community figure in the app shows.
4. Each cluster is compared with the same area over the previous 8 weeks. If that many people would be unlikely by chance (Poisson, $p < 0.01$) it is a possible outbreak; otherwise it is one to watch.

Admins see every cluster and can run a check on demand; health workers see clusters that include their centre's citizens. Either can mark a cluster as seen or turn it into a pre-filled advisory.

Layer	File
Algorithm	`model/healthnexus_ml/clustering.py` (DBSCAN, Poisson test), `model/healthnexus_ml/features.py` (syndromes)
Service and schedule	`backend/app/services/outbreaks.py` (`run`, `run_forever`), started in `backend/app/main.py` (`OUTBREAK_CLUSTERING` setting)
API	`GET /clusters`, `POST /clusters/run` (admin), `POST /clusters/{id}/acknowledge` in `backend/app/api/routes/clusters.py`
Model	`backend/app/models/community_data.py`, migration `backend/alembic/versions/0014_outbreak_clusters.py`
Page	`frontend/src/pages/Outbreaks.tsx` (map and cluster cards), client in `frontend/src/services/outbreaks.ts`
Demo data	`backend/app/seeds/seed_outbreak.py`
Tests	`backend/tests/test_outbreaks.py`, `model/tests/test_clustering.py`

7. Community advisories

Health notices with an author, an audience, an urgency and an expiry (3 to 90 days). Admins write to one district or the whole state; health workers write to their own centre's citizens. Each shows how many citizens it reaches and can be withdrawn at once. Citizens see them on their Alerts page. An advisory can cite the outbreak cluster it was written from.

Layer	File
API	`GET/POST /advisories`, `POST /advisories/{id}/withdraw` in `backend/app/api/routes/advisories.py`; citizens receive them through `GET /alerts` in `backend/app/api/routes/alerts.py`
Model	`backend/app/models/advisory.py`, migration `backend/alembic/versions/0013_advisories.py`
Page	`frontend/src/pages/Advisories.tsx`, client in `frontend/src/services/advisories.ts`; citizen display in `frontend/src/components/app/AlertList.tsx`
Tests	`backend/tests/test_advisories.py`

8. Support concerns

Citizens and workers raise concerns from Settings → Support (a wrong flag, data misuse, an access request, harassment). Admins move each through open, in review, resolved or rejected, with a note the person can read.

Layer	File
API (admin)	`GET /admin/concerns`, `PATCH /admin/concerns/{id}` in `backend/app/api/routes/admin.py`
API (raiser)	`/settings/concerns` in `backend/app/api/routes/settings.py`
Model	`backend/app/models/concern.py`
Page	`frontend/src/pages/admin/Concerns.tsx`; raising side in `frontend/src/pages/Settings.tsx` (Support tab)

9. Audit trail

Both append-only trails in one list, newest first: account actions (deactivations, registry edits, advisories, document views, research exports, examination records) and worker verification decisions (approve, reject, revoke, document accepted or rejected).

Layer	File
API	`GET /admin/audit` in `backend/app/api/routes/admin.py`
Models	`backend/app/models/audit_log.py`, `backend/app/models/phc_verification_audit_log.py`
Page	`frontend/src/pages/admin/AuditLog.tsx`

10. Research data export

A CSV of de-identified readings for public-health research, containing only citizens who turned on research use. No names, contact details, exact ages, areas or times of day; each person gets a code that changes with every download, and districts with fewer than 3 consenting people are reported as “Other”. Every download is written to the audit trail.

Layer	File
API	`GET /admin/research-export` in `backend/app/api/routes/admin.py`
Consent rule	`in_research` in `backend/app/services/consent.py`
UI	`ResearchExport` card in `frontend/src/pages/admin/AdminDashboard.tsx`, `downloadResearchExport` in `frontend/src/services/admin.ts`
Tests	`backend/tests/test_consents.py`

11. Assistant for admins

Admins can ask the assistant questions in plain words. It answers from a fixed set of read-only queries with aggregates only, never individual readings: platform overview, pending applications, load per health centre, workers with no centre, concerns summary, sign-ups, activity by district (only citizens who allow community monitoring, districts under 3 hidden) and assistant usage. With `GEMINI_API_KEY` set, Gemini picks the query and words the answer; without it, a rule-based router does.

Layer	File
Query catalog	`backend/app/services/assistant/catalog.py` (tools with `roles=ADMIN`)
Pipeline	`backend/app/services/assistant/__init__.py`, Gemini client in `backend/app/services/assistant/llm.py`
API	`backend/app/api/routes/assistant.py`
Page	`frontend/src/pages/Assistant.tsx`
Tests	`backend/tests/test_assistant.py`

12. Navigation, access and seed accounts

What	File
Admin role check	`require_admin` in `backend/app/core/dependencies.py`
Admin sidebar	`items` for `admin` in `frontend/src/components/app/AppShell.tsx`
Routes	admin `Route`s in `frontend/src/main.tsx`, paths in `frontend/src/lib/routes.ts`
First admin	`backend/app/seeds/seed_admin.py` (`ADMIN_EMAIL`, `ADMIN_PASSWORD`)
One admin per district, one worker per centre	`backend/app/seeds/seed_directory.py` (writes `backend/SEED_ACCOUNTS.md`, git-ignored)